

NotifyAfrica - Contexte global, vision produit et architecture fonctionnelle

NotifyAfrica

22 septembre 2026

1. Objet du document

Ce document donne à une équipe produit, design ou développement - humaine ou assistée par IA - le contexte complet de NotifyAfrica avant toute implémentation. Il décrit la vision, les applications, les principes de configuration, les relations entre composants, le modèle de déploiement et les règles structurantes qui doivent rester vraies dans toute la plateforme.

Il constitue la référence narrative commune aux trois applications principales : **NotifyAfrica Website**, **NotifyAfrica Console** et **NotifyAfrica Admin**.

2. Vision de NotifyAfrica

NotifyAfrica est une plateforme technologique de communication et, à terme, d'agrégation d'APIs opérateurs destinée aux entreprises, startups, fintechs, institutions et développeurs africains.

La première phase couvre les services suivants :

- SMS ;
- SMS de masse ;
- OTP ;
- WhatsApp Business ;
- Email ;
- campagnes ;
- APIs ;
- webhooks ;
- statistiques ;
- suivi de consommation ;
- wallet prépayé ;
- devis et tarification négociée.

La trajectoire produit doit permettre d'ajouter ensuite :

- APIs de collecte Mobile Money ;
- APIs de décaissement ;
- vérification de transaction ;
- paiements marchands ;
- remboursements et reversals ;
- APIs opérateurs non financières ;
- autres produits partenaires exposés via une API unifiée NotifyAfrica.

NotifyAfrica doit progressivement devenir une **couche d'abstraction et d'orchestration** entre les clients et plusieurs opérateurs ou providers.

3. Applications de l'écosystème

3.1 NotifyAfrica Website

Site public et commercial. Il présente les produits, les tarifs, les cas d'usage, la documentation, le statut des services et sert de porte d'entrée vers l'inscription et la connexion.

3.2 NotifyAfrica Console

Espace client multi-tenant. Il permet aux organisations de créer leurs projets, recharger leur wallet, consommer les produits NotifyAfrica, gérer leurs clés API, suivre leurs usages, demander des devis et gérer leurs équipes.

3.3 NotifyAfrica Admin

Backoffice interne. Il est la **source de vérité métier** pour le catalogue, les offres, les prix, les remises, les pays, les opérateurs, les providers, les routes, les devis, les wallets, les utilisateurs, les règles, les notifications et la configuration globale.

3.4 NotifyAfrica Core Backend

Backend Node.js exécuté dans un projet Next.js dédié, utilisant les Route Handlers et le runtime Node. Il centralise les règles métier, la tarification, la gestion des wallets, les appels providers, la sécurité, l'audit et les APIs consommées par les trois frontends.

3.5 Workers et traitements asynchrones

Les campagnes, retries, callbacks, webhooks, génération de rapports et tâches longues doivent être exécutés par des workers Node.js séparés du cycle HTTP, tout en partageant le même domaine métier.

4. Stack technique cible

- Node.js ;
- Next.js pour Website, Console, Admin et Core API ;
- TypeScript ;
- base relationnelle recommandée : PostgreSQL ;
- Redis pour cache, locks et queues selon besoin ;
- stockage objet compatible S3 pour pièces jointes et exports ;
- reverse proxy Nginx, Caddy ou équivalent ;
- conteneurs Docker ;
- déploiement initial sur VPS ;
- monorepo recommandé avec Turborepo ou Nx ;
- CI/CD permettant des déploiements indépendants.

5. Déploiement sur domaines distincts

Le produit final doit être déployable sur des domaines ou sous-domaines différents :

- `notifyafrica.com` - Website ;
- `app.notifyafrica.com` - Console ;
- `admin.notifyafrica.com` - Admin ;
- `api.notifyafrica.com` - Core API / Gateway ;
- `docs.notifyafrica.com` - documentation si séparée ;
- `status.notifyafrica.com` - statut de service.

Les domaines sont configurables. Le code ne doit pas supposer des URLs fixes.

6. Parcours Website vers Console

Le Website reste public, mais il doit être relié à la Console pour l'authentification.

Flux d'inscription :

1. le visiteur clique sur **Créer un compte** ;
2. le Website redirige vers la route d'inscription de la Console ;
3. la Console crée le compte utilisateur ;
4. vérification email/téléphone selon la politique ;
5. création ou rattachement à une organisation ;
6. onboarding ;
7. création du premier projet ;
8. retour dans le Dashboard Console.

Flux de connexion :

1. clic **Se connecter** sur le Website ;
2. redirection vers la Console ;
3. authentification ;

4. redirection vers le dernier contexte organisation/projet connu ou vers l'onboarding.

Le Website ne doit pas répliquer la logique d'authentification de la Console.

7. Principe de configuration dynamique

7.1 Règle fondamentale

Aucune valeur métier susceptible d'évoluer ne doit être écrite directement dans les composants du Website ou de la Console.

Cela couvre notamment :

- prix ;
- paliers ;
- marges ;
- remises ;
- taxes ;
- offres ;
- noms et descriptions de produits ;
- pays ;
- devises ;
- opérateurs ;
- providers ;
- moyens de paiement ;
- règles de devis ;
- limites ;
- quotas ;
- CTA tarifaires ;
- badges ;
- états de disponibilité ;
- paramètres OTP ;
- catégories provider ;
- contenu commercial critique.

7.2 Source de vérité

La priorité de résolution doit être :

1. configuration publiée dans NotifyAfrica Admin ;
2. configuration spécifique organisation/client ;
3. configuration spécifique projet ;
4. configuration seed initiale validée à partir du design Claude ;
5. état neutre contrôlé si aucune valeur n'est disponible.

8. Fallback issu du design Claude

Le design fourni par Claude représente également la configuration initiale fonctionnelle du produit.

Ces données ne doivent pas être copiées manuellement dans chaque frontend. Elles doivent être rassemblées dans un **seed de configuration versionné**, par exemple :

`packages/config-seed/notifyafrica.defaults.json`

Ce seed peut inclure :

- libellés initiaux ;
- produits initiaux ;
- ordres d’affichage ;
- sections marketing ;
- paliers tarifaires initiaux ;
- paramètres de formulaires ;
- valeurs d’exemple ;
- règles de visibilité ;
- paramètres visuels compatibles avec le Design System.

Au bootstrap de l’environnement, le seed peut être importé dans le Backoffice. Si le Backoffice n’a pas encore de configuration publiée pour une clé donnée, le Core API peut retourner la valeur seed correspondante avec une propriété `source = seed`.

Une fois une configuration publiée dans l’Admin, `source = admin` devient prioritaire.

9. Modèle de publication de configuration

Les configurations sensibles doivent supporter :

- Draft ;
- Review ;
- Approved ;
- Scheduled ;
- Active ;
- Archived.

Chaque modification doit être historisée avec :

- auteur ;
- date ;
- ancienne valeur ;
- nouvelle valeur ;
- motif ;
- date d’effet ;

- version.

10. Modèle commercial prépayé

NotifyAfrica fonctionne principalement en **prepaid** / **pay-as-you-go**.

Flux de base :

Recharge -> Wallet -> Estimation -> Réservation éventuelle ->
Consommation -> Débit -> Solde restant

Le wallet doit supporter :

- CREDIT ;
- DEBIT ;
- HOLD ;
- CAPTURE ;
- RELEASE ;
- REFUND ;
- ADJUSTMENT.

Le frontend ne calcule jamais seul le prix final.

11. Pricing Engine

Le Pricing Engine central est utilisé par Website, Console et Admin.

Dimensions possibles :

- produit ;
- service ;
- pays ;
- opérateur ;
- provider ;
- route ;
- catégorie ;
- volume ;
- client ;
- organisation ;
- projet ;
- devise ;
- plan ;
- contrat ;
- date d'effet ;
- promotion.

Ordre de priorité recommandé :

1. prix contractuel client ;

2. prix spécifique organisation ;
3. promotion active ;
4. remise volume ;
5. prix pays/opérateur ;
6. prix public par défaut ;
7. seed initial si aucune règle publiée.

12. Valeurs commerciales initiales

Ces valeurs sont des **données initiales** et non des constantes applicatives.

SMS :

- 0 à 25 000 SMS : 7 FCFA/SMS ;
- 25 001 à 100 000 SMS : 6,8 FCFA/SMS ;
- au-delà : tarification sur devis selon règles commerciales actives.

WhatsApp :

- coûts officiels Meta stockés comme coûts provider ;
- marge initiale NotifyAfrica : +40 % ;
- la marge est configurable ;
- les catégories, pays, prix Meta et dates d'effet sont administrables.

13. Discount Engine

Le Backoffice doit permettre des réductions :

- en pourcentage ;
- montant fixe ;
- remise unitaire ;
- prix fixe ;
- promotion temporaire ;
- remise client ;
- remise par volume ;
- bonus.

Chaque réduction définit :

- périmètre ;
- dates ;
- priorité ;
- cumulabilité ;
- plafond ;
- conditions d'éligibilité.

14. Agrégation opérateurs et API Gateway

La plateforme doit être prête à exposer une API NotifyAfrica standardisée devant plusieurs providers.

Exemple :

Client -> api.notifyafrica.com -> Gateway -> Routing -> Provider
-> Operator

Produits futurs :

- Payment Collection ;
- Payout ;
- Transaction Status ;
- Refund ;
- Reversal ;
- Account Verification ;
- Balance ;
- Merchant Payment.

15. Proxy / Gateway administrable

Le Backoffice doit permettre de gérer :

- base URLs providers ;
- endpoints ;
- méthodes HTTP ;
- auth ;
- headers ;
- timeouts ;
- transformations de requêtes ;
- transformations de réponses ;
- mapping des statuts ;
- callbacks ;
- retries ;
- priorités de route ;
- failover ;
- environnement Sandbox/Production.

Les secrets doivent être chiffrés et jamais exposés au frontend.

16. Notifications internes

Un Event & Notification Engine doit produire des événements configurables.

Événements initiaux :

- USER_REGISTERED ;

- EMAIL_VERIFIED ;
- ORGANIZATION_CREATED ;
- PROJECT_CREATED ;
- FIRST_TOPUP ;
- FIRST_API_KEY_CREATED ;
- FIRST_MESSAGE_SENT ;
- QUOTE_REQUESTED ;
- LOW_BALANCE ;
- PAYMENT_FAILED ;
- PROVIDER_DEGRADED.

Les destinataires, canaux et templates doivent être configurables dans l'Admin.

Une nouvelle inscription Console doit pouvoir déclencher immédiatement une notification Admin et/ou email à l'équipe concernée.

17. RBAC et permissions

Console :

- Owner ;
- Admin ;
- Developer ;
- Analyst ;
- Billing ;
- Viewer.

Admin :

- Super Admin ;
- Operations ;
- Finance ;
- Commercial ;
- Support ;
- Technical ;
- Risk.

Les rôles sont des ensembles de permissions granulaires.

18. Audit

Toute action sensible doit produire un audit log :

- actor ;
- action ;
- resource ;
- before ;
- after ;

- IP ;
- timestamp ;
- reason.

19. Architecture de repository recommandée

```

notifyafrica/
  apps/
    website/
    console/
    admin/
    core-api/
    worker/
  packages/
    ui/
    design-system/
    auth/
    config-client/
    config-seed/
    pricing-sdk/
    api-client/
    types/
    validation/
    observability/
  infra/
    docker/
    proxy/
    scripts/
  docs/

```

20. Exécution parallèle des trois applications

Le développement initial doit lancer en parallèle :

- Website ;
- Admin ;
- Console ;
- Core API.

Le monorepo doit proposer une commande unique de développement et des commandes ciblées par application.

Chaque application doit pouvoir être buildée et déployée indépendamment.

21. Contraintes VPS

Le premier déploiement pourra tourner sur un seul VPS suffisamment dimensionné, avec isolation par conteneurs.

Il faut toutefois préparer l'extraction progressive :

- frontends ;
- API ;
- workers ;
- base ;
- cache/queue.

L'architecture applicative ne doit pas dépendre d'un serveur unique.

22. Critères globaux d'acceptation

Le système est conforme lorsque :

- les trois applications démarrent indépendamment ;
- elles partagent les mêmes contrats API et types ;
- les prix ne sont pas définis dans les composants UI ;
- une modification publiée dans l'Admin est reflétée sur Website/Console ;
- le fallback seed fonctionne si aucune configuration Admin n'existe ;
- l'inscription Website -> Console est opérationnelle ;
- les notifications d'inscription remontent dans l'Admin ;
- l'ajout d'un nouveau produit ne nécessite pas de modifier les pages existantes ;
- l'ajout d'un nouveau provider est possible sans changer le contrat public client ;
- les domaines sont configurables par environnement ;
- les opérations sensibles sont auditées.